

Lecture 07 - Discontinuous Galerkin Methods

Part 1

Georgii Oblapenko

1 Objectives

In this lecture, we will learn about Discontinuous Galerkin methods:

1. Motivation
2. Derivation
3. Limiting, artificial viscosity

2 Motivation

We want to solve hyperbolic PDEs of the form

$$\partial_t \mathbf{u} + \nabla \cdot \mathbf{f}(\mathbf{u}) = \mathbf{s}(\mathbf{u}),$$

where $\mathbf{u}$ is the vector of conserved variables, $\mathbf{f}$ is the **flux function**, and $\mathbf{s}$ is the source term (will omit the source term for simplicity).

Finite Volume Methods allow for conservative (semi-)discretizations on very complicated geometries.

2.1 The reconstruction problem

To achieve higher-order accuracy in FVM, we use a local reconstruction of $\mathbf{u}$ in each cell, given only the values of the cell averages.

- **First-order:** $\mathbf{u} = \mathbf{u}_i$ in cell V_i
- **Second-order** (MUSCL): $\mathbf{u} = \mathbf{u}_i + \nabla \mathbf{u}_i (\mathbf{x} - \mathbf{x}_i)$
- **Higher-order:** ???

i Taylor series expansion analysis

For a smooth scalar field u :

$$u(x_{i+1/2}) = u_i + \left. \frac{\partial u}{\partial x} \right|_i \Delta x + \frac{1}{2} \left. \frac{\partial^2 u}{\partial x^2} \right|_i \Delta x^2 + \dots$$

Cell-wise constant values disregard the gradient and higher-order terms; Riemann solver adds diffusion proportional to jump at interface - so even though we omit the gradient information within the Riemann solver, we gain accuracy in the other components of the method.

2.2 ENO, WENO, TENO

- **ENO** (essentially non-oscillatory, 1987-): construct an interpolation stencil that ensures a smooth solution; but flux is at best Lipschitz-continuous w.r.t. data
- **WENO** (weighted ENO, 1994-): use intermediate stencils as well, with appropriate weights, but might still be too dissipative
- **TENO** (targeted ENO, 2016-): more close to ENO, but different stencil construction; dissipation minimization

2.3 Problems with higher-order FVM

- Memory access patterns
- Constructing the schemes on complex grids

3 Discontinuous Galerkin methods

What if we used a local (per-element) FEM approach and coupled the elements via a Finite-Volume-like flux?

$$\partial_t \mathbf{u} + \nabla \cdot \mathbf{f}(\mathbf{u}) = 0,$$

$$\Omega = \bigcup_{i=1}^N \Omega_i =: \mathcal{T}_\Omega, \quad \Omega_i \cap \Omega_j = \emptyset, \quad i \neq j.$$

Define **test function space** via **local** polynomials of degree k

$$V = \{v \in L^\infty(\Omega) : v|_{\Omega_i} \in P^k(\Omega_i) \quad \forall i = 1, \dots, N\}.$$

3.1 Weak formulation

For each scalar u , multiply by v and integrate by parts:

$$\begin{aligned} \partial_t \int_{\Omega_i} u v dx + \int_{\partial\Omega_i} \mathbf{f}(u) \cdot \mathbf{n} v dS - \int_{\Omega_i} \mathbf{f}(u) \cdot \nabla v dV &= 0, \\ \partial_t \int_{\Omega_i} u v dx + \sum_{j \in \mathcal{N}(i)} \int_{\partial\Omega_{ij}} \mathbf{f}(u) \cdot \mathbf{n}_{ij} v dS - \int_{\Omega_i} \mathbf{f}(u) \cdot \nabla v dV &= 0. \end{aligned}$$

3.2 Weak formulation

Integrate surface and volume integrals numerically via quadratures of order L and M :

$$\int_{\partial\Omega_{ij}} \mathbf{f}(u) \cdot \mathbf{n} v dS \approx \sum_{l=1}^L \omega_l \mathbf{f}(u(\mathbf{x}_{ij,l})) \cdot \mathbf{n}_{ij} v(\mathbf{x}_{ij,l}),$$

$$\int_{\Omega_i} \mathbf{f}(u) \cdot \nabla v dV \approx \sum_{m=1}^M \omega_m \mathbf{f}(u(\mathbf{x}_{i,m})) \cdot \nabla v(\mathbf{x}_{i,m}).$$

💡 Remaining parts to close the system of equations

What two parts are still missing?

📌 Remaining parts to close the system of equations

We still need to define 1) flux $\mathbf{f}(u(\mathbf{x}_{ij,l}))$ across interface and 2) define the solution u in terms of some ansatz.

3.3 Flux across elements

We need to define the flux across the interface between two elements, i.e. $\mathbf{f}(u(x_{ij,l}))$. We use the finite volume approach, i.e. we approximate $\mathbf{f}(u(\mathbf{x}_{ij,l}))$ via a *numerical flux* $\mathbf{g}(u(\mathbf{x}_{ij,l}))$:

$$\mathbf{g}(u(x_{ij,l})) = \mathbf{g}(u(\mathbf{x}_{ij,l}), u(\mathbf{x}_{ji,l})).$$

📌 Requirements for the numerical flux

- Consistency: $\mathbf{g}(u, u) = \mathbf{f}(u)$
- Lipschitz-continuity: $\|\mathbf{g}(u, v) - \mathbf{g}(u, w)\| \leq L_g \|v - w\|$

LLF/Rusanov flux (commonly used in DG):

$$g(a, b) = \frac{1}{2} (f(a) + f(b) - \lambda(b - a)), \quad \lambda = \max EV(|\partial f / \partial u|)$$

💡 Question

Why is a very diffusive flux so commonly used in DG?

3.4 Approximation of solution

Instead of arbitrary polynomials v , we switch to polynomials $v_l^{(j)}$ (of degree l) that are

- defined on the local element Ω_j
- orthogonal w.r.t each other
- form a basis of the polynomial space $P^K(\Omega_j)$

We then define the approximate solution $\hat{u}$ as

$$\hat{u} = \sum_{i=1}^N \sum_{p=0}^K u_p^{(i)} v_p^{(i)} = \sum_{i=1}^N \hat{u}^{(i)}.$$

This is the basis of the **modal** discontinuous Galerkin approach. This also allows for p -adaptivity, i.e. we can set $K = K(\Omega_j)$.

Note

We need not choose the basis polynomials to be orthogonal, but it is a convenient choice. Non-orthogonal polynomials lead to a block-structured matrix for the coefficients $u_p^{(j)}$, whereas orthogonal polynomials lead to a diagonal matrix in case of non-curved elements.

3.5 Equations for modal coefficients

Plugging in the ansatz into the weak formulation and using the orthogonality (in this case, *orthonormality*) of the basis polynomials, we obtain

$$\frac{d}{dt} u_p^{(i)} + \sum_{j \in \mathcal{N}(i)} \sum_{l=1}^L \omega_l v_p(\mathbf{x}_{ij,l}) [\mathbf{g}(u^{(i)}(\mathbf{x}_{ij,l}), u^{(j)}(\mathbf{x}_{ji,l})) \cdot \mathbf{n}_{ij}] - \sum_{m=1}^M \omega_m \mathbf{f}(u^{(i)}(\mathbf{x}_{i,m})) \cdot \nabla v_p(\mathbf{x}_{i,m}) = 0, i = 1, \dots, N.$$

Question

How do we choose orders of integration L and M ?

Question(s)

- What does DG correspond to in case of $K = 0$?
- What order of convergence do we have?

3.6 Integration order

Brief reminder:

- **Gauss-Legendre** (*GL*) quadrature: with n nodes, exact integration of polynomials on $[-1, 1]$ of degree up to $2n - 1$ ($n + 1$ points $\rightarrow$ up to degree $2n + 1$)
- **Gauss-Kronrod** (*GK*): add $n + 1$ nodes to the n Gauss-Legendre nodes, exact integration of polynomials on $[-1, 1]$ of degree up to $3n + 1$ (can reuse the GL values and obtain error estimate)
- **Clenshaw-Curtis** (*CC*): $n + 1$ nodes, exact integration of polynomials of degree up to $n + 1$, but doubling the number of nodes allows to re-use all nodes from lower order quadrature
- **Gauss-Lobatto** (*GLL*): n nodes $\rightarrow$ up to degree $2n - 3$, ($n + 1$ points $\rightarrow$ up to degree $2n - 1$); nodes include **boundary nodes** (i.e. $x_{1,n} = \mp 1$)

3.7 Integration

We need to choose order of integration L for surface fluxes through a face: - $\sum_{l=1}^L \omega_l v_p(\mathbf{x}_{ij,l}) [\mathbf{g}(u^{(i)}(\mathbf{x}_{ij,l}), u^{(j)}(\mathbf{x}_{ji,l}))]$.

And order of integration M for volume integral: - $\sum_{m=1}^M \omega_m \mathbf{f}(u^{(i)}(\mathbf{x}_{i,m})) \cdot \nabla v_p(\mathbf{x}_{i,m})$

Note that both physical and numerical fluxes $\mathbf{f}$, $\mathbf{g}$ can be non-polynomial functions of $\mathbf{u}$.

Test functions are polynomials of degree up to $K \rightarrow \nabla v_p$ has degree up to $K - 1$.

Simplest linear advection equation has flux $\mathbf{f}(u) = u\mathbf{a}$, u has degree up to K . So total degree is $2K - 1 \rightarrow$ need GL with K nodes. If non-orthogonal polynomials, also need to integrate $\int v_i v_j$ for $i, j = 1, \dots, K$: need $K + 1$ nodes.

3.8 Integration and aliasing

If we do not integrate exactly enough, we project fine oscillations (higher-frequency information) into lower-order polynomials (lower-frequency information), aka **aliasing**.

- Linear fluxes (linear advection): need at least exact integration of $2K - 1$, i.e. K nodes
- Quadratic fluxes (Burger's equation): need at least exact integration of $2K + (K - 1)$, i.e. $3K/2$ nodes
- Non-polynomial fluxes (Euler's equations): $2K + 1$ nodes or more

Even in "classical" DG, use of Gauss-Lobatto might be beneficial: we can re-use the quadrature points for the boundary terms.

4 Example: Burger's equation

1-D scalar non-linear hyperbolic equation that exhibits shocks:

$$\partial_t u + u \partial_x u = 0 \leftrightarrow \partial_t u + \frac{1}{2} \partial_x u^2 = 0$$

Let's consider $u(x, 0) = \sin(x) =: u_0(x)$, $x \in [0, 2\pi]$.

u remains constant along characteristics, i.e. along lines of constant speed. Speed at $t = 0$ at point x_0 given by $u_0(x)$; tracing backwards in time we obtain the characteristic equation

$$u(x, t) = u_0(x - u(x, t)t).$$

We can determine time at which gradient of u becomes infinite:

$$t_{shock} = -\frac{1}{\min \partial_x u_0(x)} = -\frac{1}{\min \cos(x)} = 1.$$

4.1 Analytical solution

```

1 import numpy as np
2 from scipy.optimize import fsolve
3 from matplotlib import pyplot as plt
4
5 def exact_solution_pre_shock(x_array, t):
6     """Computes exact solution for  $u_t + u u_x = 0$ ,  $u(x,0)=\sin(x)$  for  $t < 1$ ."""
7     u_exact = np.zeros_like(x_array)
8     for i, x in enumerate(x_array):
9         # Initial guess is the linear wave equation solution or just the IC
10        guess = np.sin(x)
11        # Find the root of the implicit equation
12        u_exact[i] = fsolve(lambda u: u - np.sin(x - u * t), guess)[0]
13    return u_exact
14
15 times = [0.1, 0.5, 0.999]
16 N = 50
17 x_physical = np.linspace(0, 2.0*np.pi, N)
18
19 fig = plt.figure(figsize=(8.0, 5))
20 ax = fig.add_subplot(1,1,1)
21
22 for time in times:
23     ax.plot(x_physical, exact_solution_pre_shock(x_physical, time), label=f"Analytical, t={time}")
24 ax.set_xlabel("x")
25 ax.set_ylabel("u")
26 ax.grid(':')
27 ax.legend()
28 plt.show()

```

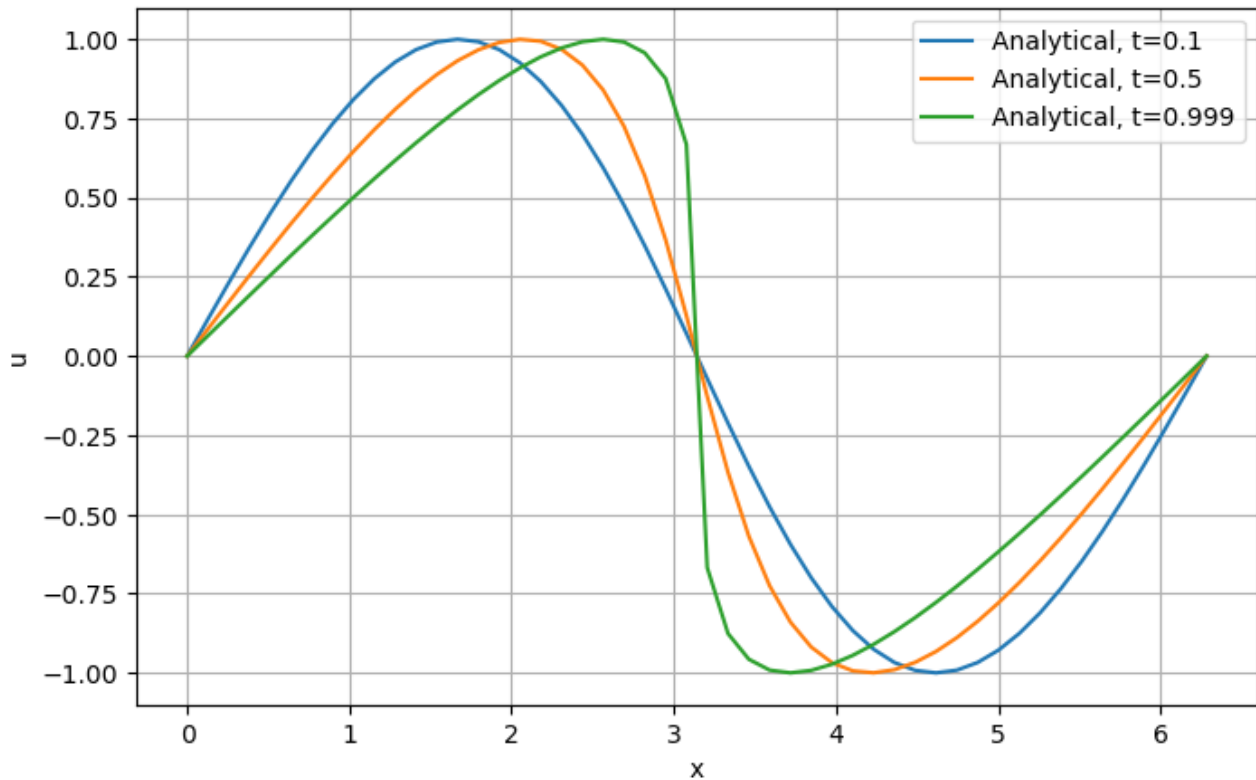


Figure 1: Analytical solution for $u(x, t)$ for $t < 1$.

4.2 Numerical solution

```

1 from numpy.polynomial.legendre import leggauss, Legendre
2
3 class RKDG1DBurgers:
4     def __init__(self, x_min, x_max, N_elements, p_degree, vol_quad_order, surf_quad_order=1):
5         """
6         1D RKDG Solver for Burgers Equation:  $u_t + (u^2/2)_x = 0$ 
7         """
8         self.x_min = x_min
9         self.x_max = x_max
10        self.N = N_elements
11        self.dx = (x_max - x_min) / self.N
12        self.p = p_degree
13
14        # Quadrature orders
15        self.vol_quad_order = vol_quad_order
16        self.surf_quad_order = surf_quad_order # Trivial in 1D, kept for API structure
17
18        # Grid setup
19        self.x_edges = np.linspace(x_min, x_max, self.N + 1)
20        self.x_centers = 0.5 * (self.x_edges[:-1] + self.x_edges[1:])
21
22        # Volume Quadrature (Gauss-Legendre)

```

```

23     self.xi_q, self.w_q = leggauss(self.vol_quad_order)
24
25     # Precompute Basis Polynomials and Derivatives at volume quadrature points
26     # P_val[k, q] = P_k(xi_q), dP_val[k, q] = P'_k(xi_q)
27     self.P_val = np.zeros((self.p + 1, self.vol_quad_order))
28     self.dP_val = np.zeros((self.p + 1, self.vol_quad_order))
29     for k in range(self.p + 1):
30         c = np.zeros(self.p + 1)
31         c[k] = 1.0
32         poly = Legendre(c)
33         self.P_val[k, :] = poly(self.xi_q)
34         self.dP_val[k, :] = poly.deriv()(self.xi_q)
35
36     # Modal coefficients matrix: shape (N, p+1)
37     self.u_hat = np.zeros((self.N, self.p + 1))
38
39     def project_initial_condition(self, u0_func):
40         """Projects continuous initial condition onto the Legendre basis."""
41         for j in range(self.N):
42             x_map = self.x_centers[j] + 0.5 * self.dx * self.xi_q
43             u0_vals = u0_func(x_map)
44
45             for k in range(self.p + 1):
46                 # Inner product: int u0(x) * P_k(xi) dxi
47                 integral = np.sum(self.w_q * u0_vals * self.P_val[k, :])
48                 self.u_hat[j, k] = ((2 * k + 1) / 2.0) * integral
49
50     def flux(self, u):
51         return 0.5 * u**2
52
53     def compute_rhs(self, u_hat):
54         """Computes the spatial discretization right-hand side L(u)."""
55         rhs = np.zeros_like(u_hat)
56
57         # 1. Evaluate solutions at element boundaries (xi = -1 and xi = 1)
58         u_left_bndy = np.zeros(self.N) # u evaluated at left edge (inner)
59         u_right_bndy = np.zeros(self.N) # u evaluated at right edge (inner)
60
61         for k in range(self.p + 1):
62             u_left_bndy += u_hat[:, k] * ((-1)**k) # P_k(-1) = (-1)^k
63             u_right_bndy += u_hat[:, k] * 1.0 # P_k(1) = 1
64
65         # 2. Compute Numerical Fluxes at interfaces (Local Lax-Friedrichs)
66         # using Periodic Boundary Conditions
67         u_minus = np.roll(u_right_bndy, shift=1) # u left of interface
68         u_plus = u_left_bndy # u right of interface
69
70         f_minus = self.flux(u_minus)
71         f_plus = self.flux(u_plus)
72         alpha = np.maximum(np.abs(u_minus), np.abs(u_plus))
73

```

```

74     f_num = 0.5 * (f_minus + f_plus - alpha * (u_plus - u_minus))
75
76     f_num_left = f_num
77     f_num_right = np.roll(f_num, shift=-1)
78
79     # 3. Compute Volume Integrals and Assemble RHS
80     for j in range(self.N):
81         # Reconstruct u at quadrature points inside cell j
82         u_q = np.zeros(self.vol_quad_order)
83         for k in range(self.p + 1):
84             u_q += u_hat[j, k] * self.P_val[k, :]
85
86         f_q = self.flux(u_q)
87
88         for k in range(self.p + 1):
89             # Volume integral term
90             vol_int = np.sum(self.w_q * f_q * self.dP_val[k, :])
91
92             # Surface flux term
93             surf_flux = f_num_right[j] * 1.0 - f_num_left[j] * ((-1)**k)
94
95             # Combine: du_hat/dt = (2k+1)/dx * (vol_int - surf_flux)
96             rhs[j, k] = ((2 * k + 1) / self.dx) * (vol_int - surf_flux)
97
98     return rhs
99
100 def solve(self, T_final, cfl=0.1):
101     """Advances the solution in time using SSP-RK3."""
102     t = 0.0
103
104     while t < T_final:
105         # Reconstruct physical values to estimate max wave speed for dt
106         u_phys = np.abs(self.u_hat[:, 0]) # Cell averages act as a proxy for speed
107         max_speed = np.max(u_phys) + 1e-8 # Avoid div by zero
108
109         # Stable time step for RKDG
110         dt = cfl * (self.dx / (2 * self.p + 1)) / max_speed
111
112         if t + dt > T_final:
113             dt = T_final - t
114
115         # SSP-RK3 Time Stepping
116         u_hat_n = self.u_hat.copy()
117
118         # Stage 1
119         L1 = self.compute_rhs(u_hat_n)
120         u_hat_1 = u_hat_n + dt * L1
121
122         # Stage 2
123         L2 = self.compute_rhs(u_hat_1)
124         u_hat_2 = 0.75 * u_hat_n + 0.25 * u_hat_1 + 0.25 * dt * L2

```

```

125
126         # Stage 3
127         L3 = self.compute_rhs(u_hat_2)
128         self.u_hat = (1.0/3.0) * u_hat_n + (2.0/3.0) * u_hat_2 + (2.0/3.0) * dt * L3
129
130         t += dt
131
132     def get_solution(self, n_points_per_cell=5):
133         """Returns physical coordinates and solution values for plotting."""
134         xi_plot = np.linspace(-1, 1, n_points_per_cell)
135         x_plot = np.zeros(self.N * n_points_per_cell)
136         u_plot = np.zeros(self.N * n_points_per_cell)
137
138         for j in range(self.N):
139             idx1 = j * n_points_per_cell
140             idx2 = (j + 1) * n_points_per_cell
141             x_plot[idx1:idx2] = self.x_centers[j] + 0.5 * self.dx * xi_plot
142
143             for k in range(self.p + 1):
144                 c = np.zeros(self.p + 1)
145                 c[k] = 1.0
146                 poly = Legendre(c)
147                 u_plot[idx1:idx2] += self.u_hat[j, k] * poly(xi_plot)
148
149         return x_plot, u_plot
150
151
152 fig = plt.figure(figsize=(8.0, 5))
153 ax = fig.add_subplot(1,1,1)
154
155 times = [0.1, 0.5, 0.999]
156
157 for time in times:
158     p = 3                                # Polynomial degree (0=FV, 1=Linear, 2=Quadratic)
159     vol_quad = 5                          # Volume quadrature order (usually p + 1)
160     surf_quad = 1                         # Surface quadrature order
161
162     # Initialize solver
163     solver = RKDG1DBurgers(x_min=0.0, x_max=2.0*np.pi, N_elements=N,
164                             p_degree=p, vol_quad_order=vol_quad, surf_quad_order=surf_quad)
165
166     # Smooth initial condition (sine wave)
167     solver.project_initial_condition(lambda x: np.sin(x))
168
169     # Solve until before shock formation (T=1.0) and after (T=1.5)
170     solver.solve(T_final=time)
171     x, u = solver.get_solution()
172     line = ax.plot(x_physical, exact_solution_pre_shock(x_physical, time), label=f"Analytical,
173     ax.plot(x, u, color=line[0].get_color(), linestyle="--", label=f"RKDG p={p}, t={time}, M={
174
175     # solver.solve(T_final=1.5)

```

```

176 # x, u = solver.get_solution()
177 # plt.plot(x, u, color='k', linestyle=":", label=f"RKDG p={p}, t={time}, M={vol_quad}")
178
179 ax.set_xlabel("x")
180 ax.set_ylabel("u")
181 ax.grid(':')
182 ax.legend()
183 plt.show()

```

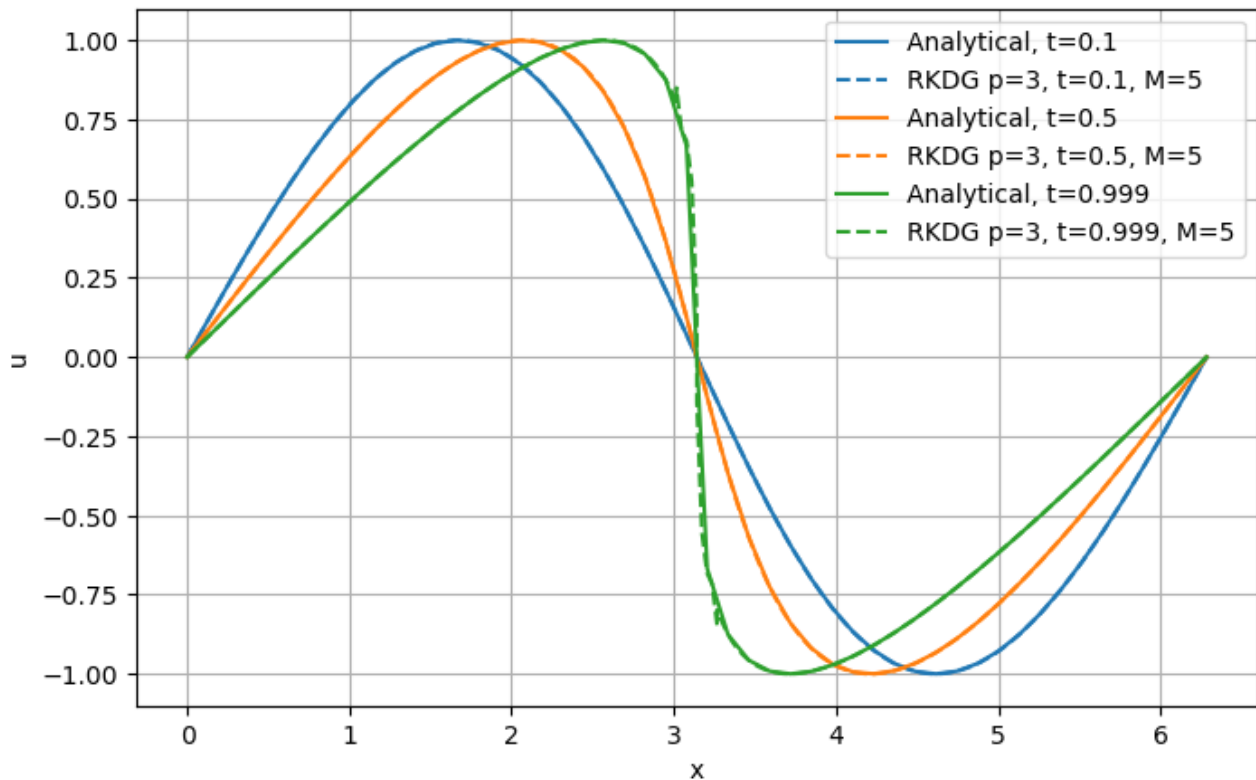


Figure 2: Analytical and numerical solutions.

5 Limiting

How do we deal in oscillations in the solution?

Slope limiting is not applicable - we have higher-order modes.

5.1 L_∞ stability/LED/TVD

(Also relevant to FVM)

- L_∞ stability: supremum/infimum of solution are bounded
- LED (local extremum diminishing): local extrema do not grow, new local extrema are not created
- TVD (total variation diminishing): $\int |\partial_x u| dx \leq \int |\partial_x u_0| dx$ (in 1D LED $\Rightarrow$ TVD)

! Goodman-Leveque obstruction theorem

A TVD numerical scheme with compact stencil for the discretization of conservation laws in space dimension $d \geq 2$ is at most first-order accurate, except in trivial cases.

i Solution to the conundrum?

Look at local extrema.

5.2 Cockburn-Shu limiter

Idea:

- project u onto piecewise linear functions
- if projection coincides with linear component u_1 of u , keep u
- otherwise
 - discard higher-order components u_p , $p \geq 2$
 - apply limiting to u_1

5.3 TVB minmod limiter

In 1D, apply

$$u_1^{(i)} = \hat{m}(u_1^{(i)}, u_0^{(i+1)} - u_0^{(i)}, u_0^{(i)} - u_0^{(i-1)}),$$
$$\hat{m}(a_1, a_2, a_3) = \begin{cases} a_1, & |a_1| \leq M\Delta x^2 \\ m(a_1, a_2, a_3), & \text{otherwise} \end{cases}$$
$$m(a_1, a_2, a_3) = \begin{cases} s \min_i |a_i|, & s := \text{sign}(a_1) = \text{sign}(a_2) = \text{sign}(a_3) \\ 0, & \text{otherwise} \end{cases}$$

💡 Question

When in the timestepping do we apply the limiting?

i Multi-dimensional extensions

Depending on element type (triangular/rectangular), limiting is based on geometric considerations and limiting based on characteristic directions (computed via diagonalization of Jacobian).

5.4 Results with limiting

```
1 class RKDG1DBurgersMinMod:
2     def __init__(self, x_min, x_max, N_elements, p_degree, vol_quad_order, surf_quad_order=1):
3         """
4         1D RKDG Solver for Burgers Equation:  $u_t + (u^2/2)_x = 0$ 
```

```

5      """
6      self.x_min = x_min
7      self.x_max = x_max
8      self.N = N_elements
9      self.dx = (x_max - x_min) / self.N
10     self.p = p_degree
11
12     # Quadrature orders
13     self.vol_quad_order = vol_quad_order
14     self.surf_quad_order = surf_quad_order # Trivial in 1D, kept for API structure
15
16     # Grid setup
17     self.x_edges = np.linspace(x_min, x_max, self.N + 1)
18     self.x_centers = 0.5 * (self.x_edges[:-1] + self.x_edges[1:])
19
20     # Volume Quadrature (Gauss-Legendre)
21     self.xi_q, self.w_q = leggauss(self.vol_quad_order)
22
23     # Precompute Basis Polynomials and Derivatives at volume quadrature points
24     # P_val[k, q] = P_k(xi_q), dP_val[k, q] = P'_k(xi_q)
25     self.P_val = np.zeros((self.p + 1, self.vol_quad_order))
26     self.dP_val = np.zeros((self.p + 1, self.vol_quad_order))
27     for k in range(self.p + 1):
28         c = np.zeros(self.p + 1)
29         c[k] = 1.0
30         poly = Legendre(c)
31         self.P_val[k, :] = poly(self.xi_q)
32         self.dP_val[k, :] = poly.deriv()(self.xi_q)
33
34     # Modal coefficients matrix: shape (N, p+1)
35     self.u_hat = np.zeros((self.N, self.p + 1))
36
37     def project_initial_condition(self, u0_func):
38         """Projects continuous initial condition onto the Legendre basis."""
39         for j in range(self.N):
40             x_map = self.x_centers[j] + 0.5 * self.dx * self.xi_q
41             u0_vals = u0_func(x_map)
42
43             for k in range(self.p + 1):
44                 # Inner product: int u0(x) * P_k(xi) dxi
45                 integral = np.sum(self.w_q * u0_vals * self.P_val[k, :])
46                 self.u_hat[j, k] = ((2 * k + 1) / 2.0) * integral
47
48     def flux(self, u):
49         return 0.5 * u**2
50
51     def compute_rhs(self, u_hat):
52         """Computes the spatial discretization right-hand side L(u)."""
53         rhs = np.zeros_like(u_hat)
54
55         # 1. Evaluate solutions at element boundaries (xi = -1 and xi = 1)

```

```

56     u_left_bndy = np.zeros(self.N) # u evaluated at left edge (inner)
57     u_right_bndy = np.zeros(self.N) # u evaluated at right edge (inner)
58
59     for k in range(self.p + 1):
60         u_left_bndy += u_hat[:, k] * ((-1)**k) # P_k(-1) = (-1)^k
61         u_right_bndy += u_hat[:, k] * 1.0      # P_k(1) = 1
62
63     # 2. Compute Numerical Fluxes at interfaces (Local Lax-Friedrichs)
64     # using Periodic Boundary Conditions
65     u_minus = np.roll(u_right_bndy, shift=1) # u left of interface
66     u_plus = u_left_bndy                    # u right of interface
67
68     f_minus = self.flux(u_minus)
69     f_plus = self.flux(u_plus)
70     alpha = np.maximum(np.abs(u_minus), np.abs(u_plus))
71
72     f_num = 0.5 * (f_minus + f_plus - alpha * (u_plus - u_minus))
73
74     f_num_left = f_num
75     f_num_right = np.roll(f_num, shift=-1)
76
77     # 3. Compute Volume Integrals and Assemble RHS
78     for j in range(self.N):
79         # Reconstruct u at quadrature points inside cell j
80         u_q = np.zeros(self.vol_quad_order)
81         for k in range(self.p + 1):
82             u_q += u_hat[j, k] * self.P_val[k, :]
83
84         f_q = self.flux(u_q)
85
86         for k in range(self.p + 1):
87             # Volume integral term
88             vol_int = np.sum(self.w_q * f_q * self.dP_val[k, :])
89
90             # Surface flux term
91             surf_flux = f_num_right[j] * 1.0 - f_num_left[j] * ((-1)**k)
92
93             # Combine: du_hat/dt = (2k+1)/dx * (vol_int - surf_flux)
94             rhs[j, k] = ((2 * k + 1) / self.dx) * (vol_int - surf_flux)
95
96     return rhs
97
98 def tvb_minmod(self, a, b, c, M):
99     """Vectorized TVB modified minmod function."""
100     # 1. TVB Check: Is the region smooth?
101     smooth = np.abs(a) <= M * self.dx**2
102
103     # 2. Standard Minmod
104     signs_match = (np.sign(a) == np.sign(b)) & (np.sign(b) == np.sign(c))
105     min_abs = np.minimum(np.abs(a), np.minimum(np.abs(b), np.abs(c)))
106     mm_val = np.sign(a) * min_abs

```

```

107     # If signs don't match, minmod returns 0
108     mm_val[~signs_match] = 0.0
109
110     # 3. Return original 'a' if smooth, otherwise limited 'mm_val'
111     return np.where(smooth, a, mm_val)
112
113
114 def apply_limiter(self, u_hat, M):
115     """Applies the hierarchical TVB minmod limiter to modal coefficients."""
116     u_hat_lim = u_hat.copy()
117
118     # Extract cell averages (mode 0)
119     u_avg = u_hat_lim[:, 0]
120
121     # Compute forward and backward differences of averages (Periodic BCs)
122     u_avg_left = np.roll(u_avg, shift=1)
123     u_avg_right = np.roll(u_avg, shift=-1)
124
125     dp = u_avg_right - u_avg
126     dm = u_avg - u_avg_left
127
128     # Extract linear slopes (mode 1)
129     u1 = u_hat_lim[:, 1]
130
131     # Apply TVB minmod to the linear slope
132     u1_limited = self.tvb_minmod(u1, dp, dm, M)
133
134     # Flag troubled cells (where the slope was actively squashed)
135     troubled = np.abs(u1 - u1_limited) > 1e-10
136
137     # Update the limited slope
138     u_hat_lim[:, 1] = u1_limited
139
140     # If p > 1, zero out higher modes ONLY in troubled cells
141     if self.p > 1:
142         for k in range(2, self.p + 1):
143             u_hat_lim[troubled, k] = 0.0
144
145     return u_hat_lim
146
147 def solve(self, T_final, cfl=0.1, M=50.0): # Added M parameter
148     """Advances the solution in time using SSP-RK3 with TVB Limiting."""
149     t = 0.0
150
151     # Initialize the limiter on the projected initial condition
152     self.u_hat = self.apply_limiter(self.u_hat, M)
153
154     while t < T_final:
155         u_phys = np.abs(self.u_hat[:, 0])
156         max_speed = np.max(u_phys) + 1e-8
157

```

```

158         dt = cfl * (self.dx / (2 * self.p + 1)) / max_speed
159
160         if t + dt > T_final:
161             dt = T_final - t
162
163         u_hat_n = self.u_hat.copy()
164
165         # Stage 1
166         L1 = self.compute_rhs(u_hat_n)
167         u_hat_1 = u_hat_n + dt * L1
168         u_hat_1 = self.apply_limiter(u_hat_1, M) # LIMIT
169
170         # Stage 2
171         L2 = self.compute_rhs(u_hat_1)
172         u_hat_2 = 0.75 * u_hat_n + 0.25 * u_hat_1 + 0.25 * dt * L2
173         u_hat_2 = self.apply_limiter(u_hat_2, M) # LIMIT
174
175         # Stage 3
176         L3 = self.compute_rhs(u_hat_2)
177         u_hat_3 = (1.0/3.0) * u_hat_n + (2.0/3.0) * u_hat_2 + (2.0/3.0) * dt * L3
178         self.u_hat = self.apply_limiter(u_hat_3, M) # LIMIT
179
180         t += dt
181
182     def get_solution(self, n_points_per_cell=5):
183         """Returns physical coordinates and solution values for plotting."""
184         xi_plot = np.linspace(-1, 1, n_points_per_cell)
185         x_plot = np.zeros(self.N * n_points_per_cell)
186         u_plot = np.zeros(self.N * n_points_per_cell)
187
188         for j in range(self.N):
189             idx1 = j * n_points_per_cell
190             idx2 = (j + 1) * n_points_per_cell
191             x_plot[idx1:idx2] = self.x_centers[j] + 0.5 * self.dx * xi_plot
192
193             for k in range(self.p + 1):
194                 c = np.zeros(self.p + 1)
195                 c[k] = 1.0
196                 poly = Legendre(c)
197                 u_plot[idx1:idx2] += self.u_hat[j, k] * poly(xi_plot)
198
199         return x_plot, u_plot
200
201     fig = plt.figure(figsize=(8.0, 5))
202     ax = fig.add_subplot(1,1,1)
203
204     times = [0.5, 2.5]
205     for time in times:
206         p = 4 # Polynomial degree (0=FV, 1=Linear, 2=Quadratic)
207         vol_quad = 6 # Volume quadrature order (usually p + 1)
208         surf_quad = 1 # Surface quadrature order

```

```

209     Mlim = 0.0
210     # Mlim2 = 8.0
211
212     # Initialize solver
213     solver = RKDG1DBurgers(x_min=0.0, x_max=2.0*np.pi, N_elements=N,
214                             p_degree=p, vol_quad_order=vol_quad, surf_quad_order=surf_quad)
215     solvermmm = RKDG1DBurgersMinMod(x_min=0.0, x_max=2.0*np.pi, N_elements=N,
216                                      p_degree=p, vol_quad_order=vol_quad, surf_quad_order=surf_q
217
218     # Smooth initial condition (sine wave)
219     solver.project_initial_condition(lambda x: np.sin(x))
220
221     # Solve until before shock formation (T=1.0) and after (T=1.5)
222     solver.solve(T_final=time)
223     x, u = solver.get_solution()
224     # line = plt.plot(x_physical, exact_solution_pre_shock(x_physical, time), label=f"Analytic
225     line = ax.plot(x, u, linestyle="--", label=f"p={p}, t={time}, M={vol_quad}")
226
227     solvermmm.project_initial_condition(lambda x: np.sin(x))
228     solvermmm.solve(T_final=time, M=Mlim)
229     x, u = solvermmm.get_solution()
230     ax.plot(x, u, linestyle="-", label=f"MinMod p={p}, t={time}, M={vol_quad}, Mlim={Mlim}")
231
232     ax.set_xlabel("x")
233     ax.set_ylabel("u")
234     ax.grid(':')
235     ax.legend(loc="lower left")
236     plt.show()

```

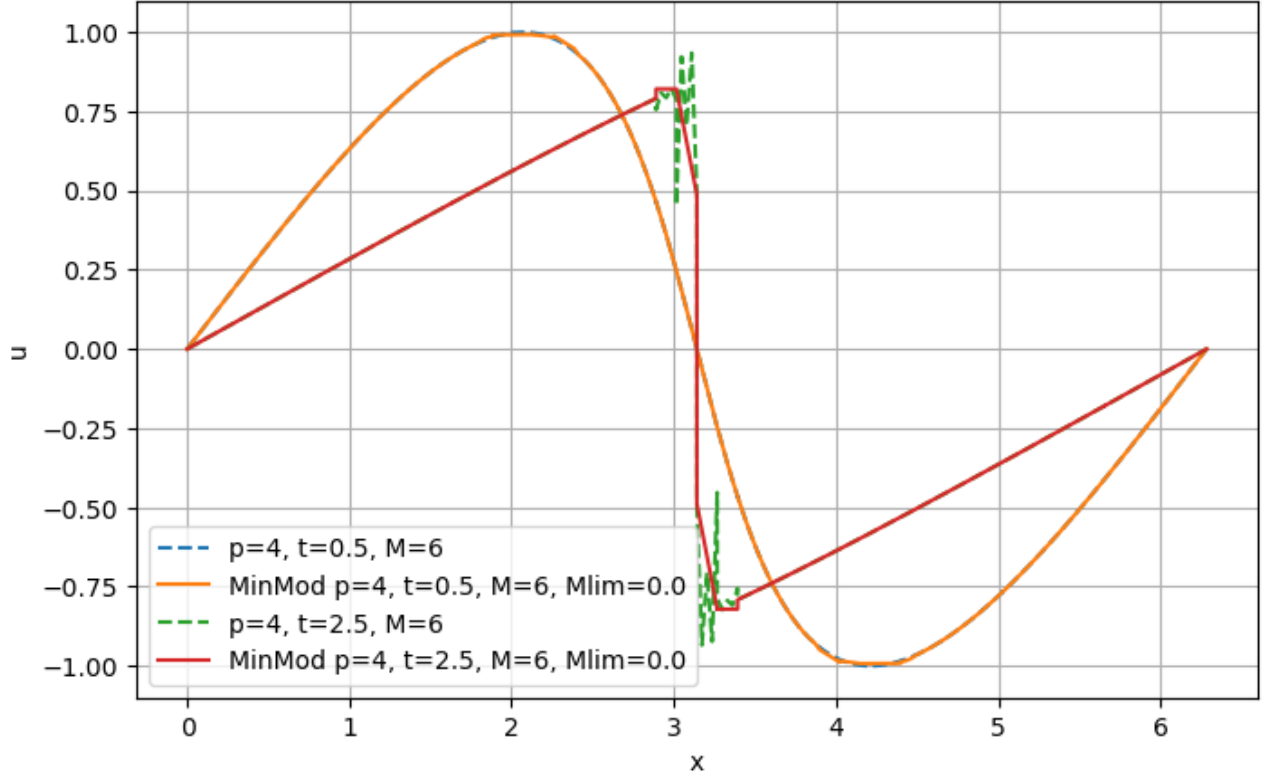


Figure 3: Analytical and numerical solutions.

5.5 Artificial viscosity (AV)

Consider viscous hyperbolic PDE:

$$\partial_t \mathbf{u} + \nabla \cdot \mathbf{f}(\mathbf{u}) - \nabla \cdot \mathbf{f}_v(\mathbf{u}, \nabla \mathbf{u}) = 0.$$

Discretization: convert to 1-st order system, use the **Bassi-Rebay (BR1, BR2)** method to discretize viscous fluxes.

For N-S,

$$f_{v,i}(\mathbf{u}, \nabla \mathbf{u}) = \mathbf{A}_{ij}^v \cdot \frac{\partial \mathbf{u}}{\partial x_j}.$$

AV:

$$f_{v,i}(\mathbf{u}, \nabla \mathbf{u}) = (\mathbf{A}_{ij}^v + \mathbf{A}_{ij}^\epsilon) \cdot \frac{\partial \mathbf{u}}{\partial x_j}.$$

5.6 AV - some considerations

- Can be formulated to conserve total enthalpy across shocks, requires change of variables from energy to enthalpy
- Can be smoothly varying across elements or piecewise-constant

Example: viscous Burger's equation:

$$\partial_t u + \frac{1}{2} \partial_x u^2 - \partial_x (\nu(x) \partial_x u) = 0.$$

For frozen time, jumps in $\nu(x)$ require jumps in $\partial_x u$.

- As always, everything is much more complicated in $D \geq 2$: high viscosity due to shock in one direction induces an error in the other directions as well.

5.7 PDE for AV

$$\partial_t \epsilon = \nabla \cdot \left(-\frac{\nabla \epsilon}{\tau} \right) + \frac{1}{\tau} \left[\frac{h(x)}{p} \lambda_{max}(\mathbf{u}) S(\mathbf{u}) - \epsilon \right],$$

Robin BC:

$$\frac{\partial \epsilon}{\partial \mathbf{n}} = \frac{\epsilon_{ref} - \epsilon}{L}; \quad \epsilon_{ref} = 0.$$

- τ ensures consistency in propagation speed between AV and original PDE
- ensures viscosity propagates only to neighbouring elements
- S is the shock indicator

Alternatively: compute per-element AV, smooth out.

6 Summary

- DG uses a local polynomial basis
- Modal DG solves for unknown coefficients in front of the polynomials
- Volume and surface terms require numerical quadratures, can be affected by aliasing
- Limiting done on linear components of the solution; limiter adjusted to avoid over-limiting

7 Questions

- Derive the weak DG formulation for a scalar conservation law
- How does orthogonality of basis functions simplify the formulation? What are the drawbacks of using orthogonal polynomials compared to monomials?
- What orders of quadrature are needed for the different parts of the semi-discretization? Which quadrature rules are used?
- (1D DG) What is the minmod limiter, and why is an additional part a_1 , $|a_1| \leq M \Delta x^2$ used? To which property of the solution u is M related?

8 Exercises

- Try changing the value of M_{lim} in the limiter, what happens? Plot error in pre-shock solution w.r.t analytical solution as function of M_{lim} for different p .
- Plot error error in pre-shock solution w.r.t analytical solution as function of integration order M for different p . Denote regions of plot where under-integration occurs.
- Implement the solver with non-orthogonal basis functions.
- Derive relation between M_{lim} and u_{xx} on a uniform grid such that a smooth extrema do not get limited. For this, assume u is smooth and both u and u_{xx} are known exactly.
- Derive a more generic form of the DG semidiscretization without the orthonormal basis assumption.

9 References

- C.-W. Shu, [Essentially non-oscillatory and weighted essentially non-oscillatory schemes](#)
- B. Cockburn, C.-W. Shu, The Runge-Kutta Discontinuous Galerkin Method for conservation laws I-V
- G.E. Barter, D.L. Darmofal, [Shock capturing with PDE-based artificial viscosity for DGFEM: Part I. Formulation](#)